Speaker notes – LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Progress meeting, about 18 minutes plus questions

How to use. One block per slide: the time budget, what to say (in your own words, not read aloud), and the one thing the audience must remember. Backup answers for questions are at the end. All numbers on the slides are measured; anything not yet run is called *planned* – say so if asked.

Suggested split: Venu slides 1-7, Bhuvan slides 8-15, Gowtham slides 16-20 and the questions.

1. Title (0:30)

Introduce yourselves by name: Venu, Bhuvan, Gowtham. Then one sentence: "We are building a fuzzer that finds bugs in PostgreSQL by generating SQL from a typed grammar, and we test whether an LLM that reads PostgreSQL's own commits makes it find more bugs."

2. Problem Statement (1:30)

- PostgreSQL adds optimiser and executor features every release. New code is the least tested code.
- Explain *wrong-result bug*: the database returns rows, no error, but the rows are wrong. These are dangerous because nobody notices.
- Existing generators: either random (mostly invalid SQL, so they never reach the optimiser) or hand-written (good, but someone has to add every new feature by hand).
- Read the research question on the slide.

Remember: valid SQL that exercises *new* features, with oracles that catch *silent* errors.

3. Motivation (1:15)

- PostgreSQL is everywhere; correctness matters.
- Name two or three recent features from the slide and say they all change how queries are planned (e.g. OR-to-ANY rewrites `x = 1 OR x = 2` into `x = ANY('{1,2}')`).
- We fuzz `master` (version 20devel). Bugs found there are regressions in fresh code; developers fix them before release and credit the reporter.
- LLMs have read the docs and can read a commit; a fuzzer cannot.

4. Limits of existing approaches (1:15)

Walk the table top to bottom, one sentence per row. End with the gap: we want SQL that is valid, replayable (so mutations stay valid), and extensible by an LLM – without calling the LLM for every query, which would be slow and expensive.

If asked about citations: "we will verify every citation before the paper".

5. Our idea in one picture (1:00)

Three arms, same budget, same server, same schema, same oracle. Arm 1 is the baseline (random and mutation). Arm 2 is our typed grammar with tree-based mutation. Arm 3 adds the LLM, which edits the grammar and writes seed test cases. The comparison is the experiment. The next slide is what is ours.

6. Three claims (1:00)

Read the three claims slowly. Do not add a fourth.

1. One tree is used to generate, mutate, rewrite and reduce. That is why mutations and metamorphic rewrites stay valid: types and scopes are re-checked, not hoped for.
2. We will not say "sound". We will say the number: valid-statement rate for the generator, triage precision for the oracle.
3. The LLM does not write the paper's numbers. A proposal counts only after the server runs it and the changed lines of that commit are covered.

If asked "is this just SQLancer plus ChatGPT?", this slide is the answer.

7. Architecture (1:00)

Follow the arrows: generator produces a test case; executor runs it on a PostgreSQL built from source with assertions; the oracle classifies the outcome; signatures deduplicate; metrics and reports come out. Only the generator changes between arms – that is what makes the comparison fair.

8. Typed grammar and replay mutation (1:45)

- We wrote our own grammar language because a plain context-free grammar produces SQL that parses but fails in the analyser (unknown columns, type errors, aggregates in `WHERE`).
- Our rules carry SQL types and scopes; grouping rules are context flags.
- Course link: generating an expression of type T is building a typing derivation $\Gamma \vdash e : T$ (type systems lecture).
- Each statement keeps its derivation tree. Mutation swaps a subtree; replay walks the tree again and re-binds names and types in the new context, so the mutant is still valid. Point at the three panels of the figure.

Remember: most generated statements reach the planner, and mutations do not break validity.

9. Oracle stack (1:30)

- Level 1: crashes and assertion failures (`TRAP:` lines in the server log) – always bugs.
- Level 2: internal errors (`XX000`, "should not happen" messages).
- Level 3: wrong results. Today: run the same query under 17 planner settings inside savepoints and compare results as multisets. Planned: metamorphic oracles – TLP, NoREC and equivalent rewrites applied directly on our derivation trees.
- False-positive control: only deterministic queries (no `LIMIT`, no `random()`, no order-dependent aggregates), floats masked, re-runs.

10. LLM loop (planned) (1:30)

- Say clearly: this is the plan for the next two weeks.
- The LLM reads recent commits and their regression tests, proposes grammar rules and seeds.
- Key design point: the **validation gate**. Nothing the LLM writes enters the fuzzer unless it parses, runs on the server and covers new changed lines. The LLM proposes, the server and coverage decide.
- Diff coverage tells us per commit whether the LLM helped.

11. Experimental design (1:30)

Read the four RQs. Stress RQ3: we revert 8 to 12 historical bug fixes to get a server with known bugs, then measure how fast each arm finds them. This gives ground truth even if new bugs are rare (it is the mutation-testing idea from the course, applied to fuzzers).

12. Statistics and setup (1:00)

Fuzzing is random, so one run proves nothing. 10 trials per arm, Mann-Whitney U for significance, Vargha-Delaney $\hat{A}_{12}$ for effect size (0.5 = no difference). Coverage is collected on a separate build so it does not slow down the arms. Every bug candidate goes through the same triage pipeline.

13. Status so far (1:00)

Only measured numbers. Highlights:

- PostgreSQL built from `master` with assertions; our acceptance test passes 10/10.
- 65 tests passed when the harness and grammar landed. After the random/mutation baseline was added, the suite is 84 passed.
- Two engineering findings: reading the server log only after a failure made the executor faster (0.99 s to 0.15 s for 20 test cases); using `127.0.0.1` instead of `localhost` cut connection time from 131 s to 0.2 s on Windows/WSL.
- The random arm is no longer a stand-in. It builds small SQL trees and then mutates the text, on the same schema as the grammar.
- Point at the next slide for the server numbers. Do not say "we have not measured validity" any more.

14. What the server accepted (1:15)

Walk the three bars. All three are 300 test cases, seed 1, the grammar arm, the same assert-enabled server.

- Round 1: 83.4 percent of 812 statements. It did not start until we moved an index off a virtual generated column. PostgreSQL refuses that index.
- Round 2: 83.1 percent. The rate barely moved, but semantic errors fell from 76 to 39. The run got slower (83 seconds) because `generate_series` could use a huge column as its end.
- Round 3: 88.4 percent, 821 of 929 statements, 56 seconds. Semantic errors are 28. Six statements still hit the timeout.
- Round 4: **100 percent, 966 of 966 statements, 22.1 seconds.** The error list is empty. The differential oracle checked 230 queries and ran 460 variants. No mismatch.
- Say this clearly: **zero bug candidates**, including on the clean sample. Every statement ran. Nothing crashed, nothing asserted, and no two planner settings disagreed. That is why the 4 hour runs are next.

If asked how the last jump happened: an aggregate no longer looks at columns from outside its own query; a join builds a real equality between its two sides; inserts respect NOT NULL, keys, the numeric check, and foreign keys; MERGE is fed one row so it cannot touch a row twice; JSON functions are given objects; ranges are built low-to-high. Two schema changes are part of the same result, and say them: `t0` is no longer seeded with `NaN` and infinities, and the range-partitioned table has a DEFAULT partition, so a row cannot miss every partition.

15. Who does what (0:45)

Say the names, then the jobs, in this order:

- Venu and Bhuvan read what landed in PostgreSQL recently: MERGE, SQL/JSON, generated columns, partitions, and new planner rewrites. They extend the grammar so those paths actually run.
- Bhuvan is also the first person on a bug candidate: reproduce it, shrink it, and decide whether it is worth a report.
- Gowtham keeps the comparison experiment, the statistics, the LLM arm, and the paper moving so the grade does not wait on a crash.

One sentence to close: the servers stay busy while the feature list is being read.

16. Expected outcomes (1:00)

Say the split out loud. RQ1, RQ3 and RQ4 will exist whether or not we find a new bug. The extra target is one bug the developers acknowledge, which is why we file in week 2. If that acknowledgement does not arrive, we report zero new bugs and the candidates we rejected. Say explicitly that we do not treat that as full marks on the 50 % implementation component.

17. Draft paper structure (0:45)

Point at the five graded parts: motivating example, workflow, research questions, related work, threats. If there is no new bug, the motivating example is a re-injected historical bug with the measured detection times.

18. Timeline (0:45)

Point at the meetings, the presentation on 12 November and the paper on 13 November. Campaigns run continuously from this week.

19. Risks and asks (1:00)

Main risk: no new bug in time. The mitigation is early filing, not a claim that the other results replace the bug. Ask the professor whether a mailing-list acknowledgement, before a fix commit, counts, and write the answer down.

20. Thank you / questions

Gowtham takes the questions, and either of the others answers the part they own. Venu and Bhuvan take grammar, server, and feature questions. Gowtham takes the experiment, the statistics, and the paper.

Backup answers

Why not SQLancer? SQLancer has excellent oracles but hand-written generators per DBMS; we add an editable typed grammar that an LLM can extend, and we reuse its oracle ideas (TLP, NoREC) on derivation trees.

Why build PostgreSQL from source? Assertions and debug symbols turn silent corruption into `TRAP:` messages and give backtraces; release builds have assertions off.

How do you know a wrong result is real? Determinism filter, re-run, compare with all planner variants, reduce to a few lines, then reason from the SQL standard and the docs before reporting.

Is the LLM necessary? That is exactly RQ4: we measure with and without it, including token cost.

What counts as a unique bug? A unique signature (stack, assertion, normalised message, or plan variant plus shape), confirmed by manual triage.

What if no new bug is found? The report and the meetings do not need one: RQ1, RQ3 and RQ4 are complete, and the motivating example becomes a re-injected bug with measured times. The top of the 50 % implementation component still needs a developer acknowledgement, so campaigns and filing continue until the deadline. We will not pretend a missing bug is full marks.

Is AI use allowed? Yes, per the updated course instructions; we disclose it in the paper.

You already have 100 percent valid. Why no bug? Three hundred test cases is under half a minute of server time (22.1 seconds). Bugs in a database this size show up over hours. 100 percent means every

statement in that sample reached execution, and the differential oracle checked 230 of the queries. Zero candidates is the honest count. The four-hour runs are the next measurement.

Why split the team that way? New code is where an unknown bug is most likely, so Venu and Bhuvan stay on recent features. A candidate in a log is useless until someone shrinks it, so Bhuvan does that first. The paper and the comparison still have to exist if the bug is late, so Gowtham owns them.